SET 10

PART A — Short questions and answers

Q1. Explain why PNR is selected as the key attribute of the BOOKING entity.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q2. Compare DDL and DML commands with one example each.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q3. Explain how removing a transitive dependency brings a relation into Third Normal Form (3NF).

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q4. Interpret the query: SELECT TrainName FROM TRAIN WHERE TrainNo IN (SELECT TrainNo FROM BOOKING);

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q5. Describe the function of SUM applied to the Fare column.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q6. Describe the purpose of an explicit cursor.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q7. Compare a graph database and a relational database for storing station connections.

Answer: A graph database stores entities as nodes and their connections as relationships.

Q8. Compare the deleteOne() and deleteMany() methods.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q9. Interpret the MongoDB query: db.trains.find({Source: "Chennai"}).limit(3)

Answer: The query returns all documents that satisfy the specified filter condition.

Q10. Interpret the Cypher query: MATCH (s:Station {name: 'Chennai'})- [:CONNECTED_TO]->(t:Station) RETURN t.name;

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

PART B — System solution

Scenario: A railway reservation office maintains a Railway Reservation System to store passenger details, train

1(a) ER design:

• **PASSENGER**(PassengerID, PassengerName, Age, Gender); primary key: **PassengerID**.

• **TRAIN**(TrainNo, TrainName, Source, Destination, Fare); primary key: **TrainNo**.

• **BOOKING**(PNR, PassengerID, TrainNo, JourneyDate, SeatNo, ClassType); primary key: **PNR**.

Connect the two main entity tables through the transaction/relationship table. The transaction table has its own primary key and foreign keys to both entity tables. Cardinality: each entity can participate in many transactions; each transaction belongs to one row from each entity table (1:M on both sides, resolving M:N). Include the transaction's date/status/amount attributes shown in the question.

1(b) SQL DDL/DML: Create each table with its listed columns, PRIMARY KEY on its ID, and FOREIGN KEYs in the transaction table. Add NOT NULL to required fields; use CHECK/UNIQUE/DEFAULT where the question asks. Insert five sample rows per table, making sure foreign-key values exist.

```
CREATE TABLE PASSENGER (
  PassengerID NUMBER PRIMARY KEY,
  PassengerName VARCHAR2(100),
  Age NUMBER,
  Gender VARCHAR2(100)
```

```
);
CREATE TABLE TRAIN (
  TrainNo VARCHAR2(100) PRIMARY KEY,
  TrainName VARCHAR2(100),
  Source VARCHAR2(100),
  Destination VARCHAR2(100),
  Fare VARCHAR2(100)
);
CREATE TABLE BOOKING (
  PNR VARCHAR2(100) PRIMARY KEY,
  PassengerID NUMBER,
  TrainNo VARCHAR2(100),
  JourneyDate DATE,
  SeatNo VARCHAR2(100),
  ClassType VARCHAR2(100)
);
```

2(a) Join query: Select the two entity names/details from the transaction table by joining the transaction table to both entity tables using their IDs.

```
SELECT a.*, b.*
FROM BOOKING x
JOIN PASSENGER a ON x.PassengerID = a.PassengerID
JOIN TRAIN b ON x.TrainNo = b.TrainNo;
```

2(b) Aggregate query: Group the transaction rows by the requested category/type and apply HAVING to the aggregate condition stated in the question.

```
SELECT grouping_column, COUNT(*) AS total
FROM transaction_table
GROUP BY grouping_column
HAVING COUNT(*) > threshold;
```

2(c) PL/SQL procedure: Create a procedure with an IN parameter for the requested ID. Join the transaction table to the related entity tables, filter by that parameter, and print selected values using DBMS_OUTPUT.PUT_LINE.

```
CREATE OR REPLACE PROCEDURE show_details(p_id IN NUMBER) AS
BEGIN
  FOR r IN (
    SELECT a.*, b.*, x.*
    FROM transaction_table x
    JOIN entity_table_1 a ON x.entity1_id = a.entity1_id
    JOIN entity_table_2 b ON x.entity2_id = b.entity2_id
    WHERE x.transaction_id = p_id
  ) LOOP
    DBMS_OUTPUT.PUT_LINE('Details retrieved');
  END LOOP;
END;
/
```

3. Graph database (Neo4j/Cypher): Create the required node labels and relationship types exactly as specified in the set. Example pattern:

```
CREATE (a:EntityA {id: 1, name: 'Name A'});
CREATE (b:EntityB {id: 2, name: 'Name B'});
MATCH (a:EntityA {id: 1}), (b:EntityB {id: 2})
CREATE (a)-[:RELATED_TO]->(b);
MATCH (a:EntityA)-[:RELATED_TO]->(b:EntityB)
RETURN a.name, b.name;
```